Develop code that serializes and/or de-serializes objects using the following APIs from **java.io**: **DataInputStream**, **DataOutputStream**, **FileInputStream**, **FileOutputStream**, **ObjectInputStream**, **ObjectOutputStream** and **Serializable**.

DataInputStream

A data input stream lets an application read primitive Java data types from an underlying input stream in a machine-independent way. An application uses a data output stream to write data that can later be read by a data input stream.

A `DataInputStream` takes as a constructor an `InputStream`.

Some methods:

```
public final boolean readBoolean() throws IOException
public final byte readByte() throws IOException
public final int readUnsignedByte() throws IOException
public final short readShort() throws IOException
public final int readUnsignedShort() throws IOException
public final char readChar() throws IOException
public final int readInt() throws IOException
public final long readLong() throws IOException
public final float readFloat() throws IOException
public final double readDouble() throws IOException
```

```
FileInputStream fis = new FileInputStream("test.txt");
DataInputStream dis = new DataInputStream(fis);
boolean value = dis.readBoolean();
```

DataOutputStream

A data output stream lets an application write primitive Java data types to an output stream in a portable way. An application can then use a data input stream to read the data back in.

A `DataOutputStream` takes as a constructor an `OutputStream`.

Some methods:

```
public final void writeBoolean(boolean v) throws IOException
public final void writeByte(int v) throws IOException
public final void writeShort(int v) throws IOException
public final void writeChar(int v) throws IOException
public final void writeInt(int v) throws IOException
public final void writeLong(long v) throws IOException
public final void writeFloat(float v) throws IOException
public final void writeDouble(double v) throws IOException
public final void writeBytes(String s) throws IOException
public final void writeChars(String s) throws IOException
```

FileInputStream

A FileInputStream obtains input bytes from a file in a file system. What files are available depends on the host environment.

FileInputStream is meant for reading streams of raw bytes such as image data. For reading streams of characters, consider using FileReader.

Some methods:

```
// Reads a byte of data from this input stream
public int read() throws IOException
public int read(byte[] b) throws IOException
public int read(byte[] b, int off, int len) throws IOException
```

```
FileInputStream fis = new FileInputStream("test.txt");
```

FileOutputStream

A file output stream is an output stream for writing data to a File or to a FileDescriptor. Whether or not a file is available or may be created depends upon the underlying platform. Some platforms, in particular, allow a file to be opened for writing by only one FileOutputStream (or other file-writing object) at a time. In such situations the constructors in this class will fail if the file involved is already open.

FileOutputStream is meant for writing streams of raw bytes such as image data. For writing streams of characters, consider using FileWriter.

Some methods:

```
// Writes the specified byte to this file output stream
public void write(int b) throws IOException
public void write(byte[] b) throws IOException
public void write(byte[] b, int off, int len) throws IOException
```

```
FileOutputStream fos = new FileOutputStream("testout.txt");
```

```
// append
FileOutputStream fos = new FileOutputStream("testout.txt", true);
```

ObjectInputStream

An ObjectInputStream deserializes primitive data and objects previously written using an ObjectOutputStream. ObjectOutputStream and ObjectInputStream can provide an application with persistent storage for graphs of objects when used with a FileOutputStream and FileInputStream respectively. ObjectInputStream is used to recover those objects previously serialized. Other uses include passing objects between hosts using a socket stream or for marshaling and unmarshaling arguments and parameters in a remote communication system.

ObjectInputStream ensures that the types of all objects in the graph created from the stream match the classes present in the Java Virtual Machine. Classes are loaded as required using the standard mechanisms.

Only objects that support the java.io.Serializable or java.io.Externalizable interface can be read from streams.

The method readObject is used to read an object from the stream. Java's safe casting should be used to get the desired type. In Java, strings and arrays are objects and are treated as objects during serialization. When read they need to be cast

to the expected type.

Primitive data types can be read from the stream using the appropriate method on `DataInput`.

The default deserialization mechanism for objects restores the contents of each field to the value and type it had when it was written. Fields declared as `transient` or `static` are IGNORED by the deserialization process. References to other objects cause those objects to be read from the stream as necessary. Graphs of objects are restored correctly using a reference sharing mechanism. New objects are always allocated when deserializing, which prevents existing objects from being overwritten.

Reading an object is analogous to running the constructors of a new object. Memory is allocated for the object and initialized to zero (NULL). No-arg constructors are invoked for the non-serializable classes and then the fields of the serializable classes are restored from the stream starting with the serializable class closest to `java.lang.Object` and finishing with the object's most specific class:

```
public class A {
    public int aaa = 111;

    public A() {
        System.out.println("A");
    }
}
```

Class `B` extends non-serializable class `A`:

```
import java.io.Serializable;

public class B extends A implements Serializable {

    public int bbb = 222;

    public B() {
        System.out.println("B");
    }
}
```

The client code:

```
public class Client {
    public static void main(String[] args) throws Exception {

        B b = new B();
        b.aaa = 888;
        b.bbb = 999;

        System.out.println("Before serialization:");
        System.out.println("aaa = " + b.aaa);
        System.out.println("bbb = " + b.bbb);

        ObjectOutputStream save = new ObjectOutputStream(new FileOutputStream("datafile"));
        save.writeObject(b); // Save object
        save.flush(); // Empty output buffer


        ObjectInputStream restore = new ObjectInputStream(new FileInputStream("datafile"));
        B z = (B) restore.readObject();

        System.out.println("After deserialization:");
        System.out.println("aaa = " + z.aaa);
        System.out.println("bbb = " + z.bbb);
```

```
    }
}
```

The client's output:

```
A
B
Before serialization:
aaa = 888
bbb = 999
A
After deserialization:
aaa = 111
bbb = 999
```

For example to read from a stream as written by the example in `ObjectOutputStream`:

```
FileInputStream fis = new FileInputStream("test.tmp");
ObjectInputStream ois = new ObjectInputStream(fis);

int i = ois.readInt();
String today = (String) ois.readObject();
Date date = (Date) ois.readObject();

ois.close();
```

Classes control how they are serialized by implementing either the `java.io.Serializable` or `java.io.Externalizable` interfaces.

Implementing the `Serializable` interface allows object serialization to save and restore the entire state of the object and it allows classes to evolve between the time the stream is written and the time it is read. It automatically traverses references between objects, saving and restoring entire graphs.

Serializable classes that require special handling during the serialization and deserialization process should implement the following methods:

```
private void writeObject(java.io.ObjectOutputStream stream) throws IOException;
private void readObject(java.io.ObjectInputStream stream) throws IOException,
        ClassNotFoundException;
```

The `readObject` method is responsible for reading and restoring the state of the object for its particular class using data written to the stream by the corresponding `writeObject` method. The method does not need to concern itself with the state belonging to its superclasses or subclasses. State is restored by reading data from the `ObjectInputStream` for the individual fields and making assignments to the appropriate fields of the object. Reading primitive data types is supported by `DataInput`.

Serialization does not read or assign values to the fields of any object that does not implement the `java.io.Serializable` interface. Subclasses of Objects that are not serializable can be serializable. In this case the non-serializable class must have a no-arg constructor to allow its fields to be initialized. In this case it is the responsibility of the subclass to save and restore the state of the non-serializable class. It is frequently the case that the fields of that class are accessible (`public`, package, or `protected`) or that there are get and set methods that can be used to restore the state.

Implementing the `Externalizable` interface allows the object to assume complete control over the contents and format of the object's serialized form. The methods of the `Externalizable` interface, `writeExternal` and `readExternal`, are called to save and restore the objects state. When implemented by a class they can write and read their own state using all of the methods of `ObjectOutput` and `ObjectInput`. It is the responsibility of the objects to handle any versioning that occurs.

Enum constants are deserialized differently than ordinary serializable or externalizable objects. The serialized form of an enum constant consists solely of its name; field values of the constant are not transmitted. To deserialize an enum constant, ObjectInputStream reads the constant name from the stream; the deserialized constant is then obtained by calling the static method Enum.valueOf(Class, String) with the enum constant's base type and the received constant name as arguments. Like other serializable or externalizable objects, enum constants can function as the targets of back references appearing subsequently in the serialization stream. The process by which enum constants are deserialized CANNOT be customized: any class-specific readObject, readObjectNoData, and readResolve methods defined by enum types are ignored during deserialization.

ObjectOutputStream

An ObjectOutputStream writes primitive data types and graphs of Java objects to an OutputStream. The objects can be read (reconstituted) using an ObjectInputStream. Persistent storage of objects can be accomplished by using a file for the stream. If the stream is a network socket stream, the objects can be reconstituted on another host or in another process.

Only objects that support the java.io.Serializable interface can be written to streams. The class of each serializable object is encoded including the class name and signature of the class, the values of the object's fields and arrays, and the closure of any other objects referenced from the initial objects.

The method writeObject is used to write an object to the stream. Any object, including Strings and arrays, is written with writeObject. Multiple objects or primitives can be written to the stream. The objects MUST be read back from the corresponding ObjectInputStream with the SAME types and in the SAME order as they were written.

Primitive data types can also be written to the stream using the appropriate methods from DataOutput. Strings can also be written using the writeUTF method.

The default serialization mechanism for an object writes the class of the object, the class signature, and the values of all non-transient and non-static fields. References to other objects (except in transient or static fields) cause those objects to be written also. Multiple references to a single object are encoded using a reference sharing mechanism so that graphs of objects can be restored to the same shape as when the original was written.

For example to write an object that can be read by the example in ObjectInputStream:

```
FileOutputStream fos = new FileOutputStream("test.tmp");
ObjectOutputStream oos = new ObjectOutputStream(fos);

oos.writeInt(12345);
oos.writeObject("Today");
oos.writeObject(new Date());

oos.close();
```

Classes that require special handling during the serialization and deserialization process must implement special methods with these exact signatures:

```
private void readObject(java.io.ObjectInputStream stream)
        throws IOException, ClassNotFoundException;
private void writeObject(java.io.ObjectOutputStream stream)
        throws IOException
```

The writeObject method is responsible for writing the state of the object for its particular class so that the corresponding readObject method can restore it. The method does not need to concern itself with the state belonging to the object's superclasses or subclasses. State is saved by writing the individual fields to the ObjectOutputStream using the writeObject method or by using the methods for primitive data types supported by DataOutput.

Serialization does not write out the fields of any object that does not implement the java.io.Serializable interface. Subclasses of Objects that are not serializable can be serializable. In this case the non-serializable class must have a no-arg constructor to allow its fields to be initialized. In this case it is the responsibility of the subclass to save and restore the state

of the non-serializable class. It is frequently the case that the fields of that class are accessible (`public`, package, or `protected`) or that there are get and set methods that can be used to restore the state.

Serializable

Serializability of a class is enabled by the class implementing the `java.io.Serializable` interface. Classes that do not implement this interface will not have any of their state serialized or deserialized. All subtypes of a serializable class are themselves serializable. The serialization interface has NO methods or fields and serves only to identify the semantics of being serializable.

To allow subtypes of non-serializable classes to be serialized, the subtype may assume responsibility for saving and restoring the state of the supertype's `public`, `protected`, and (if accessible) package fields. The subtype may assume this responsibility only if the class it extends has an accessible no-arg constructor to initialize the class's state. It is an error to declare a class `Serializable` if this is not the case. The error will be detected at runtime.

During deserialization, the fields of non-serializable classes will be initialized using the `public` or `protected` no-arg constructor of the class. A no-arg constructor must be accessible to the subclass that is serializable. The fields of serializable subclasses will be restored from the stream.

When traversing a graph, an object may be encountered that does not support the `Serializable` interface. In this case the `NotSerializableException` will be thrown and will identify the class of the non-serializable object.

Classes that require special handling during the serialization and deserialization process must implement special methods with these exact signatures:

```
private void writeObject(java.io.ObjectOutputStream out) throws IOException
private void readObject(java.io.ObjectInputStream in) throws IOException,
        ClassNotFoundException;
```

Serialization

Imagine a graph of objects that lead from the object to be saved. The entire graph must be saved and restored:

```
Obj 1 --> Obj 2 --> Obj 3
      \--> Obj 4 --> Obj 5
                \ --> Obj 6
```

We need a byte-coded representation of objects that can be stored in a file external to Java programs, so that the file can be read later and the objects can be reconstructed. Serialization provides a mechanism for saving and restoring objects.

Serializing an object means to code it as an ordered series of bytes in such a way that it can be rebuilt (really a copy) from that byte stream. Deserialization generates a new live object graph out of the byte stream.

The serialization mechanism needs to store enough information so that the original object can be recreated including all objects to which it refers (the object graph).

Java has classes (in the `java.io` package) that allow the creation of streams for object serialization and methods that write to and read from these streams.

Only an object of a class that implements the EMPTY interface `java.io.Serializable` or a subclass of such a class can be serialized.

What is saved:

- The class of the object.
- The class signature of the object.

- All instance variables NOT declared `transient`.
- Objects referred to by non-transient instance variables.

If a duplicate object occurs when traversing the graph of references, only ONE copy is saved, but references are coded so that the duplicate links can be restored.

Saving an object (an array of **Fruit**)

1. Open a file and create an `ObjectOutputStream` object.

```
ObjectOutputStream save = new ObjectOutputStream(new FileOutputStream("datafile"));
```

2. Make `Fruit` serializable:

```
class Fruit implements Serializable
```

3. Write an object to the stream using `writeObject()`.

```
Fruit [] fa = new Fruit[3];
// Create a set of 3 Fruits and place them in the array.
...
save.writeObject(fa); // Save object (the array)
save.flush(); // Empty output buffer
```

Restoring the object

1. Open a file and create an `ObjectInputStream` object.

```
ObjectInputStream restore = new ObjectInputStream(new FileInputStream("datafile"));
```

2. Read the object from the stream using `readObject()` and then cast it to its appropriate type.

```
Fruit[] newFa;
// Restore the object:
newFa = (Fruit[])restore.readObject();
```

or

```
Object ob = restore.readObject();
```

When an object is retrieved from a stream, it is validated to ensure that it can be rebuilt as the intended object. Validation may fail if the class definition of the object has changed.

A class whose objects are to be saved must implement interface `Serializable`, with no methods, or the `Externalizable` interface, with two methods. Otherwise, runtime exception will be thrown:

```
Exception in thread "main" java.io.NotSerializableException: Bag
        at java.io.ObjectOutputStream.writeObject0(Unknown Source)
```

```
        at java.io.ObjectOutputStream.writeObject(Unknown Source)
        at Client.main(Client.java:17)
```

The first superclass of the class (maybe `Object`) that is not serializable must have a no-parameter constructor.

The class must be visible at the point of serialization.

The `implements Serializable` clause acts as a tag indicating the possibility of serializing the objects of the class.

All primitive types are serializable.

Transient fields (with `transient` modifier) are NOT serialized, (i.e., not saved or restored).

A class that implements `Serializable` must mark `transient` fields of classes that do not support serialization (e.g., a file stream).

Because the deserialization process will create new instances of the objects. Comparisons based on the "`==`" operator MAY NO longer be valid.

Main saving objects methods:

```
public ObjectOutputStream(OutputStream out) throws IOException
public final void writeObject(Object obj) throws IOException
public void flush() throws IOException
public void close() throws IOException
```

Main restoring objects methods:

```
public ObjectInputStream(InputStream in) throws IOException, SecurityException
public final Object readObject() throws IOException, ClassNotFoundException
public void close() throws IOException
```

`ObjectOutputStream` and `ObjectInputStream` also implement the methods for writing and reading primitive data and `Strings` from the interfaces `DataOutput` and `DataInput`, for example:

```
writeBoolean(boolean b)    <==> boolean readBoolean()
writeChar(char c)          <==> char readChar()
writeInt(int i)            <==> int readInt()
writeDouble(double d)      <==> double readDouble()
```

No methods or class variables are saved when an object is serialized.

A class knows which methods and static data are defined in it.

Serialization example (client class):

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
```

```
public class Client {
    public static void main(String ... aaa) throws IOException, ClassNotFoundException {
        Bag b = new Bag();
        Bag a = null;

        ObjectOutputStream save = new ObjectOutputStream(new FileOutputStream("datafile"));
        System.out.println("Before serialization:");
        System.out.println(b);
        save.writeObject(b); // Save object
        save.flush(); // Empty output buffer

        ObjectInputStream restore = new ObjectInputStream(new FileInputStream("datafile"));
        a = (Bag) restore.readObject();
        System.out.println("After deserialization:");
        System.out.println(a);
    }
}
```

If the Bag class does not implement Serializable:

```
import java.util.Arrays;

public class Bag {

        Fruit[] fruits = new Fruit[3];

        public Bag() {
                fruits[0] = new Fruit("Orange");
                fruits[1] = new Fruit("Apple");
                fruits[2] = new Fruit("Pear");
        }

        public String toString() {
                return "Bag of fruits :" + Arrays.toString(fruits);
        }
}
```

We get the following runtime exception:

```
Before serialization:
Bag of fruits :[Fruit : Orange, Fruit : Apple, Fruit : Pear]
Exception in thread "main" java.io.NotSerializableException: Bag
        at java.io.ObjectOutputStream.writeObject0(Unknown Source)
        at java.io.ObjectOutputStream.writeObject(Unknown Source)
        at Client.main(Client.java:17)
```

Now, make Bag class implement Serializable, but Fruit class still is not Serializable:

```
public class Fruit {

        String name = "";

        public Fruit(String name) {
                this.name = name;
        }

        public String toString() {
                return "Fruit : " + name;
        }
```

```
}
```

We get same exception but in class `Fruit`:

```
Before serialization:
Bag of fruits :[Fruit : Orange, Fruit : Apple, Fruit : Pear]
Exception in thread "main" java.io.NotSerializableException: Fruit
        at java.io.ObjectOutputStream.writeObject0(Unknown Source)
        at java.io.ObjectOutputStream.writeArray(Unknown Source)
        at java.io.ObjectOutputStream.writeObject0(Unknown Source)
        ...
```

We can ask not to serialize `Fruit` classes by making array of fruits `transient`:

```
import java.io.Serializable;
import java.util.Arrays;

public class Bag implements Serializable {

        transient Fruit[] fruits = new Fruit[3]; // do not save to disk

        public Bag() {
                fruits[0] = new Fruit("Orange");
                fruits[1] = new Fruit("Apple");
                fruits[2] = new Fruit("Pear");
        }

        public String toString() {
                return "Bag of fruits :" + Arrays.toString(fruits);
        }
}
```

Now the program is running without exceptions, but fruits are not restoring in the bag after deserialization:

```
Before serialization:
Bag of fruits :[Fruit : Orange, Fruit : Apple, Fruit : Pear]
After deserialization:
Bag of fruits :null
```

Only when both `Bag` and `Fruits` are serializable, we get expected output:

```
import java.io.Serializable;
import java.util.Arrays;

public class Bag implements Serializable {

        Fruit[] fruits = new Fruit[3];

        public Bag() {
                fruits[0] = new Fruit("Orange");
                fruits[1] = new Fruit("Apple");
                fruits[2] = new Fruit("Pear");
        }

        public String toString() {
                return "Bag of fruits :" + Arrays.toString(fruits);
```

```
        }
}
```

```
import java.io.Serializable;

public class Fruit implements Serializable {
        String name = "";

        public Fruit(String name) {
                this.name = name;
        }

        public String toString() {
                return "Fruit : " + name;
        }
}
```

The client's output now will be:

```
Before serialization:
Bag of fruits :[Fruit : Orange, Fruit : Apple, Fruit : Pear]
After deserialization:
Bag of fruits :[Fruit : Orange, Fruit : Apple, Fruit : Pear]
```

NOTE, `static` and `transient` fields are NOT serialized:

```
import java.io.Serializable;

public class MyClass implements Serializable {
        transient int one;
        private int two;
        static int three;

        public MyClass() {
                one = 1;
                two = 2;
                three = 3;
        }

        public String toString() {
                return "one : " + one + ", two: " + two + ", three: " + three;
        }
}
```

This code saves class instance:

```
...
MyClass b = new MyClass();
ObjectOutputStream save = new ObjectOutputStream(new FileOutputStream("datafile"));
save.writeObject(b); // Save object
save.flush(); // Empty output buffer
...
```

Deserialize:

```
...
MyClass a = null;
ObjectInputStream restore = new ObjectInputStream(new FileInputStream("datafile"));
a = (MyClass) restore.readObject();
System.out.println("After deserialization:");
System.out.println(a);
...
```

The output:

```
After deserialization:
one : 0, two: 2, three: 0
```

As you can see, `static` and `transient` fields got the default values for instance variables.

Prev

Given a scenario involving navigating file systems, reading from files, or writing to files, develop the correct solution using the following classes (sometimes in combination), from java.io: BufferedReader, BufferedWriter, File, FileReader, FileWriter and PrintWriter.

Up

Home

Next

Use standard J2SE APIs in the java.text package to correctly format or parse dates, numbers, and currency values for a specific locale; and, given a scenario, determine the appropriate methods to use if you want to use the default locale or a specific locale. Describe the purpose and use of the java.util.Locale class.